

Week 5 Exam Study Guide

Strings + Functions with def and for loops

How to use this:

- Part 1: Cheat sheet. The 6 loop patterns + if/elif rules. Read once.
- Part 2: Worksheet. Problems with writing lines. Write answers on your Kindle with the pen. No peeking.
- Part 3: Answer key. Only after you finish a round.
- Do 3 random problems cold (like the exam). Redo wrong ones tomorrow morning from a blank page.

Before you check each answer, ask:

- Did I use return (not print)? Only output_even prints.
- Is total = 0 / result = "" ABOVE the loop?
- Did I write range(1, num + 1) with the + 1?
- Is return True AFTER the loop (isogram)?
- Did I check "unknown" / bad length BEFORE the loop?
- Colons after for / if / elif / else? Used == not =?

PART 1: Cheat Sheet

1. Build a string letter by letter

Used in: reverse_string

```
result = ""
for letter in word:
    result = letter + result    # letter in FRONT = reversed
    # result = result + letter  # letter at END = same order
return result
```

2. Loop over words

Used in: first_letters, last_letters

```
result = ""
for word in sentence.split():
    result = result + word[0]    # word[-1] = last letter
return result
```

3. Loop by index

Used in: hamming_distance, skip_letter_first, skip_letter_second

```
if len(str1) != len(str2):
    return "The strings must be the same length"
distance = 0
for i in range(len(str1)):
    if str1[i] != str2[i]:
        distance = distance + 1
return distance

for i in range(0, len(word), 2):    # indexes 0,2,4 (first)
for i in range(1, len(word), 2):    # indexes 1,3,5 (second)
```

4. Early return

Used in: is_isogram

```
for letter in word:
    if word.count(letter) > 1:
        return False
return True    # AFTER the loop, not inside
```

5. Add up a number range

Used in: square_sum, cube_sum, odd_sum

```
if num < 1:
    return "unknown"          # check FIRST
total = 0
for i in range(1, num + 1):   # +1 because range stops before end
    total = total + i ** 2     # ** 3 for cube
return total

# odd_sum: for n in range(smaller, larger + 1):
#             if n % 2 == 1: total = total + n
```

6. Build a list / repeated text

Used in: find_factors, design_rug, output_even

```
factors = []
for i in range(1, num + 1):
    if num % i == 0:
        factors.append(i)
return factors

rug = "Your rug is:"
for row in range(length):
    rug = rug + "\n" + pattern * width
return rug
```

If / elif rules

- "Ignore case" means: first line `x = x.lower()`
- Check the invalid / most specific case FIRST. else goes LAST.
- `elif` = another outcome of the SAME decision. Separate ifs = independent checks.
- `and` = both true. `or` = either true. Range check: `20 <= age <= 39`
- `=` assigns. `==` compares. Colon after every `if` / `elif` / `else` / `for` / `while`.
- Return the EXACT text from the question (capitals, spaces, punctuation).

String / math tricks

<code>word[0]</code>	first letter	<code>word[-1]</code>	last letter
<code>word[:3]</code>	first 3 letters	<code>temp[:-1]</code>	all but last
<code>temp[-1]</code>	last character	<code>word.count("a")</code>	times "a" appears
<code>"hello big".split()</code> -> <code>["hello", "big"]</code>			
<code>int(x * 100) / 100</code> truncate to 2 decimals (not round)			
<code>n % 2 == 0</code>	even	<code>n % 2 == 1</code>	odd
		<code>num % i == 0</code>	i divides num
<code>a // b</code>	whole-number divide	<code>a % b</code>	remainder

While loop pattern (sum_loop, create_word)

```
ask ONCE before the loop
while not-the-stop-condition:
    use the answer
    ask AGAIN at the BOTTOM of the loop
return result
```

PART 2: Worksheet

Write each function from scratch on the lines. Include def and return.

Round 1: Strings + for loop (warm-up)

#1 reverse_string(word)

Return the word backwards.

"abc" -> "cba"

"python" -> "nohtyp"

#2 first_letters(sentence)

First letter of every word, joined.

"hello big world" -> "hbw"

#3 last_letters(sentence)

Last letter of every word, joined.

"hello big world" -> "ogd"

#4 skip_letter_first(word)

Letters at index 0, 2, 4, ...

"abcdef" -> "ace"

"abcde" -> "ace"

#5 skip_letter_second(word)

Letters at index 1, 3, 5, ...

"abcdef" -> "bdf"

"a" -> ""

#6 is_isogram(word)

True if no letter repeats, else False.

"python" -> True

"hello" -> False

"" -> True

Round 2: Harder strings

#7 hamming_distance(str1, str2)

Count positions where the letters differ.

If lengths differ, return the message below.

"cat", "cut" -> 1

"abc", "xyz" -> 3

"ab", "abc" -> "The strings must be the same length"

#8 flip_flop(word)

Swap first half and last half. Middle letter stays if length is odd.

"abcd" -> "cdab"

"abcde" -> "decab"

"a" -> "a"

#9 get_drink_ID(flavor, capacity)

First 3 letters of flavor + capacity.

("lemonade", 12) -> "lem12"

#10 is_fever(temp)

temp like "99F" or "37C" (unit may be lowercase).

True if F > 98.6 or C > 37, else False.

"99F" -> True

"37C" -> False

"98.6F" -> False

#11 is_boiling(temp)

temp like "212F" or "100c".

True if F >= 212 or C >= 100, else False.

"212F" -> True

"99c" -> False

"100C" -> True

Round 3: Number loops

#12 square_sum(num)

$1^2 + 2^2 + \dots + \text{num}^2$.

If num < 1 return "unknown".

3 -> 14

1 -> 1

0 -> "unknown"

#13 cube_sum(num)

$1^3 + 2^3 + \dots + \text{num}^3$.

If num < 1 return "unknown".

3 -> 36

-2 -> "unknown"

#14 odd_sum(smaller_num, larger_num)

Sum of odd numbers from smaller to larger, inclusive.

(1, 10) -> 25

(2, 2) -> 0

(4, 9) -> 21

#15 find_factors(num)

List of all factors of num, ascending.

6 -> [1, 2, 3, 6]

7 -> [1, 7]

#16 design_rug(width, length, pattern)

Return "Your rug is:" then `length` lines, each `pattern` repeated `width` times.

(3, 2, "***") -> "Your rug is:\n***\n***"

#17 output_even(smaller_num, larger_num)

PRINT (don't return) each even number, one per line, inclusive.

(1, 6) prints 2, 4, 6

Round 4: Bonus (same patterns, not in your files)

#18 count_vowels(word)

How many vowels (aeiou).

"banana" -> 3

"xyz" -> 0

#19 double_letters(word)

Each letter twice.

"abc" -> "aabbcc"

#20 remove_spaces(sentence)

Remove all spaces.

"a b c" -> "abc"

#21 factorial(n)

$1 * 2 * \dots * n$. 0 -> 1.

5 -> 120

0 -> 1

#22 count_evens(smaller_num, larger_num)

How many even numbers, inclusive.

(1, 10) -> 5

#23 multiples(n, count)

First `count` multiples of n as a list.

(3, 4) -> [3, 6, 9, 12]

#24 largest_letter_count(word)

How many times the most common letter appears.

"banana" -> 3

Round 5: if / elif

#25 access_rights(user_role)

Ignore case.

admin -> "full", user -> "limited", guest -> "view", else "unknown".

"Admin" -> "full"

"bob" -> "unknown"

#26 traffic_light(light_color)

Ignore case.

green -> "Go", yellow -> "Yield", red -> "Stop", else "Invalid light color".

"Green" -> "Go"

"blue" -> "Invalid light color"

#27 `serve_coffee(selected_coffee)`

Ignore case. espresso / latte / cappuccino -> "Here is your latte!"

Else -> "Sorry, we don't have mocha." (lowercase name in both)

"LATTE" -> "Here is your latte!"

"MOCHA" -> "Sorry, we don't have mocha."

#28 `check_letter(letter)`

Single lowercase letter. Return "Vowel" or "Consonant".

"a" -> "Vowel"

"z" -> "Consonant"

#29 find_winner(player1, player2)

Rock paper scissors, ignore case. Rock beats scissors, scissors beats paper, paper beats rock.

Return "It's a tie!", "Player 1 wins!" or "Player 2 wins!".

```
("rock","scissors") -> "Player 1 wins!"  
("Paper","scissors") -> "Player 2 wins!"  
("rock","ROCK") -> "It's a tie!"
```

#30 triangle_type(side_1, side_2, side_3)

All equal -> "equilateral". Exactly two equal -> "isosceles". None equal -> "scalene".

(3, 3, 3) -> "equilateral"

(3, 4, 3) -> "isosceles"

(3, 4, 5) -> "scalene"

#31 count_duplicates(num_1, num_2, num_3)

All same -> "You entered the same number 3 times"

Two same -> "You entered the same number 2 times"

Else -> "Each number is unique"

(1, 1, 1) -> 3 times

(1, 2, 1) -> 2 times

(1, 2, 3) -> unique

#32 pool_time(grade, time)

grade is "k" or int 1-12. time is "morning"/"afternoon" (ignore case).

k-3: 9 AM / 1 PM 4-8: 10 AM / 2 PM 9-12: 11 AM / 3 PM

Anything else -> "unknown".

```
("k", "Morning") -> "9 AM"
```

```
(8, "Afternoon") -> "2 PM"
```

```
(13, "morning") -> "unknown"
```

```
(5, "night") -> "unknown"
```

#33 resting_rate(age, athl_goal)

athl_goal is "Above Average" / "Below Average" (ignore case).

Age 20-39: "47-72" above, "73-93" below

Age 40-59: "46-71" above, "72-94" below

Age 60-79: "45-70" above, "71-97" below

Otherwise -> "unknown".

```
(25, "Above Average") -> "47-72"
```

```
(59, "Below Average") -> "72-94"
```

```
(19, "Above Average") -> "unknown"
```

```
(30, "average") -> "unknown"
```

#34 highway_directions(highway_num)

Valid: 1-999 and last two digits not 00.

Invalid -> "I-100 is an invalid highway number".

Last-two-digits odd -> "I-95 runs north/south". Even -> "I-10 runs east/west".

Hint: highway_num % 100

95 -> "I-95 runs north/south"

110 -> "I-110 runs east/west"

100 -> invalid

1000 -> invalid

Round 6: Math + while loops

#35 leg_counter(chickens, cows, pigs)

Chicken 2 legs, cow 4, pig 4. Return total.

(1, 1, 1) -> 10

#36 battery_counter(e_dolls, rc_cars, robo_dogs)

Dolls 2 batteries, cars 4, dogs 6. Return total.

(2, 0, 3) -> 22

#37 pyramid_volume(b, h)

$(b^2 * h) / 3$, truncated (not rounded) to 2 decimals.

(2.5, 7) -> 14.58

#38 cone_volume(r, h)

$\pi * r^2 * h / 3$, truncated to 2 decimals. Use import math, math.pi.

(3, 5) -> 47.12

```
#39 convert_bronze(bronze_coins)
```

No loop. 20 bronze = 1 silver, 15 silver = 1 gold.

Like "1 gold 2 silver 5 bronze". Skip zero parts. 0 -> "0 bronze".

```
345 -> "1 gold 2 silver 5 bronze"
```

```
20 -> "1 silver"
```

```
25 -> "1 silver 5 bronze"
```

#40 sum_loop()

Ask input("Enter an integer: ") repeatedly, add each number.
Stop at the first negative number (don't add it). Return total.

inputs 5, 10, 3, -1 -> 18

#41 create_word()

Ask input() for letters until user types "done". Return all letters joined.

inputs c, a, t, done -> "cat"

PART 3: Answer Key

Only look after you finish. Numbers match the worksheet.

#1 reverse_string

```
def reverse_string(word):  
    result = ""  
    for letter in word:  
        result = letter + result  
    return result
```

#2 first_letters

```
def first_letters(sentence):  
    result = ""  
    for word in sentence.split():  
        result = result + word[0]  
    return result
```

#3 last_letters

```
def last_letters(sentence):  
    result = ""  
    for word in sentence.split():  
        result = result + word[-1]  
    return result
```

#4 skip_letter_first

```
def skip_letter_first(word):  
    result = ""  
    for i in range(0, len(word), 2):  
        result = result + word[i]  
    return result
```

#5 skip_letter_second

```
def skip_letter_second(word):  
    result = ""  
    for i in range(1, len(word), 2):  
        result = result + word[i]  
    return result
```

#6 is_isogram

```
def is_isogram(word):
    for letter in word:
        if word.count(letter) > 1:
            return False
    return True
```

#7 hamming_distance

```
def hamming_distance(str1, str2):
    if len(str1) != len(str2):
        return "The strings must be the same length"
    distance = 0
    for i in range(len(str1)):
        if str1[i] != str2[i]:
            distance = distance + 1
    return distance
```

#8 flip_flop

```
def flip_flop(word):
    half = len(word) // 2
    first_half = word[:half]
    middle = word[half:len(word) - half]
    second_half = word[len(word) - half:]
    return second_half + middle + first_half
```

#9 get_drink_ID

```
def get_drink_ID(flavor, capacity):
    return flavor[:3] + str(capacity)
```

#10 is_fever

```
def is_fever(temp):
    unit = temp[-1].upper()
    number = float(temp[:-1])
    if unit == "F":
        return number > 98.6
    else:
        return number > 37
```

#11 is_boiling

```
def is_boiling(temp):
    unit = temp[-1].upper()
    number = float(temp[:-1])
    if unit == "F":
        return number >= 212
    else:
        return number >= 100
```

#12 square_sum

```
def square_sum(num):
    if num < 1:
        return "unknown"
    total = 0
    for i in range(1, num + 1):
        total = total + i ** 2
    return total
```

#13 cube_sum

```
def cube_sum(num):
    if num < 1:
        return "unknown"
    total = 0
    for i in range(1, num + 1):
        total = total + i ** 3
    return total
```

#14 odd_sum

```
def odd_sum(smaller_num, larger_num):
    total = 0
    for n in range(smaller_num, larger_num + 1):
        if n % 2 == 1:
            total = total + n
    return total
```

#15 find_factors

```
def find_factors(num):
    factors = []
    for i in range(1, num + 1):
        if num % i == 0:
            factors.append(i)
    return factors
```

#16 design_rug

```
def design_rug(width, length, pattern):  
    rug = "Your rug is:"  
    for row in range(length):  
        rug = rug + "\n" + pattern * width  
    return rug
```

#17 output_even

```
def output_even(smaller_num, larger_num):  
    for n in range(smaller_num, larger_num + 1):  
        if n % 2 == 0:  
            print(n)
```

#18 count_vowels

```
def count_vowels(word):  
    count = 0  
    for letter in word:  
        if letter in "aeiou":  
            count = count + 1  
    return count
```

#19 double_letters

```
def double_letters(word):  
    result = ""  
    for letter in word:  
        result = result + letter * 2  
    return result
```

#20 remove_spaces

```
def remove_spaces(sentence):  
    result = ""  
    for letter in sentence:  
        if letter != " ":  
            result = result + letter  
    return result
```

#21 factorial

```
def factorial(n):  
    result = 1  
    for i in range(1, n + 1):  
        result = result * i  
    return result
```

#22 count_evens

```
def count_evens(smaller_num, larger_num):  
    count = 0  
    for n in range(smaller_num, larger_num + 1):  
        if n % 2 == 0:  
            count = count + 1  
    return count
```

#23 multiples

```
def multiples(n, count):  
    result = []  
    for i in range(1, count + 1):  
        result.append(n * i)  
    return result
```

#24 largest_letter_count

```
def largest_letter_count(word):  
    best = 0  
    for letter in word:  
        if word.count(letter) > best:  
            best = word.count(letter)  
    return best
```

#25 access_rights

```
def access_rights(user_role):  
    role = user_role.lower()  
    if role == "admin":  
        return "full"  
    elif role == "user":  
        return "limited"  
    elif role == "guest":  
        return "view"  
    else:  
        return "unknown"
```

#26 traffic_light

```
def traffic_light(light_color):
    color = light_color.lower()
    if color == "green":
        return "Go"
    elif color == "yellow":
        return "Yield"
    elif color == "red":
        return "Stop"
    else:
        return "Invalid light color"
```

#27 serve_coffee

```
def serve_coffee(selected_coffee):
    coffee = selected_coffee.lower()
    if coffee == "espresso" or coffee == "latte" or coffee == "cappuccino":
        return "Here is your " + coffee + "!"
    else:
        return "Sorry, we don't have " + coffee + "."
```

#28 check_letter

```
def check_letter(letter):
    if letter in "aeiou":
        return "Vowel"
    else:
        return "Consonant"
```

#29 find_winner

```
def find_winner(player1, player2):
    player1 = player1.lower()
    player2 = player2.lower()
    if player1 == player2:
        return "It's a tie!"
    elif (player1 == "rock" and player2 == "scissors") or \
        (player1 == "scissors" and player2 == "paper") or \
        (player1 == "paper" and player2 == "rock"):
        return "Player 1 wins!"
    else:
        return "Player 2 wins!"
```

#30 triangle_type

```
def triangle_type(side_1, side_2, side_3):
    if side_1 == side_2 and side_2 == side_3:
        return "equilateral"
    elif side_1 == side_2 or side_1 == side_3 or side_2 == side_3:
        return "isosceles"
    else:
        return "scalene"
```

#31 count_duplicates

```
def count_duplicates(num_1, num_2, num_3):
    if num_1 == num_2 and num_2 == num_3:
        return "You entered the same number 3 times"
    elif num_1 == num_2 or num_1 == num_3 or num_2 == num_3:
        return "You entered the same number 2 times"
    else:
        return "Each number is unique"
```

#32 pool_time

```
def pool_time(grade, time):
    if grade == "k" or 1 <= grade <= 3:
        morning = "9 AM"
        afternoon = "1 PM"
    elif 4 <= grade <= 8:
        morning = "10 AM"
        afternoon = "2 PM"
    elif 9 <= grade <= 12:
        morning = "11 AM"
        afternoon = "3 PM"
    else:
        return "unknown"
    if time.lower() == "morning":
        return morning
    elif time.lower() == "afternoon":
        return afternoon
    else:
        return "unknown"
# "k" must be checked first with `or`
```

#33 resting_rate

```
def resting_rate(age, athl_goal):
    if 20 <= age <= 39:
        above = "47-72"
        below = "73-93"
    elif 40 <= age <= 59:
        above = "46-71"
        below = "72-94"
    elif 60 <= age <= 79:
        above = "45-70"
        below = "71-97"
    else:
        return "unknown"
    goal = athl_goal.lower()
    if goal == "above average":
        return above
    elif goal == "below average":
        return below
    else:
        return "unknown"
```

#34 highway_directions

```
def highway_directions(highway_num):
    if highway_num < 1 or highway_num > 999 or highway_num % 100 == 0:
        return "I-" + str(highway_num) + " is an invalid highway number"
    primary = highway_num % 100
    if primary % 2 == 1:
        return "I-" + str(highway_num) + " runs north/south"
    else:
        return "I-" + str(highway_num) + " runs east/west"
```

#35 leg_counter

```
def leg_counter(chickens, cows, pigs):
    return chickens * 2 + cows * 4 + pigs * 4
```

#36 battery_counter

```
def battery_counter(e_dolls, rc_cars, robo_dogs):
    return e_dolls * 2 + rc_cars * 4 + robo_dogs * 6
```

#37 pyramid_volume

```
def pyramid_volume(b, h):
    volume = (b ** 2 * h) / 3
    return int(volume * 100) / 100
```

#38 cone_volume

```
import math

def cone_volume(r, h):
    volume = math.pi * (r ** 2 * h) / 3
    return int(volume * 100) / 100
```

#39 convert_bronze

```
def convert_bronze(bronze_coins):
    gold = bronze_coins // 300          # 20 * 15 = 300
    bronze_coins = bronze_coins % 300
    silver = bronze_coins // 20
    bronze = bronze_coins % 20
    result = ""
    if gold > 0:
        result = result + str(gold) + " gold "
    if silver > 0:
        result = result + str(silver) + " silver "
    if bronze > 0:
        result = result + str(bronze) + " bronze"
    if result == "":
        return "0 bronze"
    return result.strip()
```

#40 sum_loop

```
def sum_loop():
    total = 0
    number = int(input("Enter an integer: "))
    while number >= 0:
        total = total + number
        number = int(input("Enter an integer: "))
    return total
```

#41 create_word

```
def create_word():
    word = ""
    letter = input("Enter a letter (or type done): ")
    while letter != "done":
        word = word + letter
        letter = input("Enter a letter (or type done): ")
    return word
```